How OpenShift IPI Installation Works on AWS (Backend Explained)

This document explains what happens **behind the scenes** when you run:

```bash
./openshift-install create cluster --dir ~/ocp-ha-install --log-level=info
```

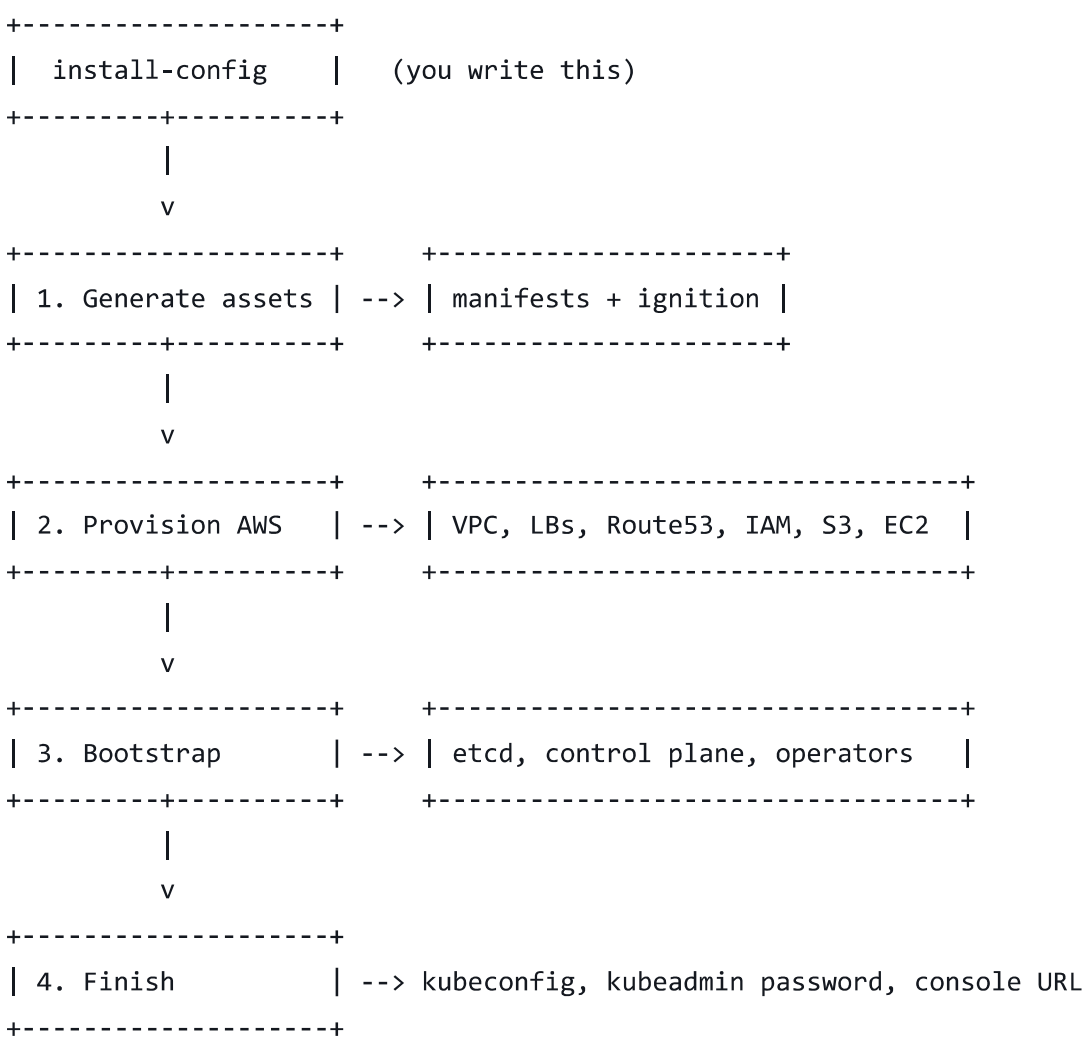
It covers the installer's phases, the key components, and how the bootstrap process builds the cluster.

---

1. Big picture

The `openshift-install` program is an **orchestrator**. It does not run the cluster itself. It does four jobs:

1. **Generates configuration** (manifests and Ignition files).
2. **Creates AWS infrastructure** (network, load balancers, DNS, IAM, EC2 instances).
3. **Waits for the cluster to bootstrap itself** using a temporary bootstrap machine.
4. **Cleans up** the bootstrap machine and prints the login details.


install-config
(you write this)
1. Generate assets
--> manifests + ignition
2. Provision AWS
--> VPC, LBs, Route53, IAM, S3, EC2
3. Bootstrap
--> etcd, control plane, operators
4. Finish
--> kubeconfig, kubeadmin password, console URL

---

2. Key components and terms

Term	What it is	Why it matters
install-config.yaml	Your input file (cluster name, region, machine types, pull secret, SSH key)	Installer reads it. It is consumed during install, so keep a backup.
Manifests	Kubernetes YAML files generated from install-config	Define cluster network, DNS, ingress, machine sets, etc.
Ignition config	JSON file that tells a machine how to configure itself on first boot	Used by RHCOS. Three files are created: `bootstrap.ign`, `master.ign`, `worker.ign`.
RHCOS (Red Hat CoreOS)	Immutable OS image used for all cluster nodes	Configured by Ignition on first boot, and managed by the cluster after install.
Bootstrap node	Temporary machine that starts the first control plane	Created at the start and deleted at the end.
Machine Config Server (MCS)	Service on the bootstrap and then on control plane nodes, port **22623**	Serves Ignition configs to new nodes during boot.
kube-apiserver	Kubernetes API, port **6443**	Main entry point for `oc` and nodes.
etcd	Cluster database storing all Kubernetes state	Needs 3 members on control plane nodes for HA.
Cluster Version Operator (CVO)	Operator that applies the release payload and tracks cluster version	Rolls out all cluster operators in order.
Cluster operators	Operators for networking, DNS, ingress, auth, console, MCO, etc.	Each reports Available / Degraded / Progressing.
Machine Config Operator (MCO)	Manages OS configuration of nodes using MachineConfig objects	Can reboot nodes when config changes.
Machine API	Creates and manages cloud VMs using MachineSets	Creates the worker EC2 instances after the control plane is up.
CSR (Certificate Signing Request)	Node asks the cluster for a certificate to join	Must be approved for nodes to become Ready.

Term	What it is	Why it matters
kubeconfig	Login file for `oc` (`auth/kubeconfig`)	Created at the end of install.
kubeadmin	Temporary admin user	Password in `auth/kubeadmin-password`. Replace with a real identity provider.

---

3. Phase by phase

Phase 0: Input check

The installer:

- Reads `install-config.yaml` and validates fields (base domain, region, instance types, pull secret format, SSH key).
- Checks AWS credentials and permissions.
- Checks that the Route 53 public hosted zone for the base domain exists.

If anything is wrong, the install stops here. No AWS resources are created yet.

Phase 1: Generate manifests and Ignition configs

The installer creates the Kubernetes objects and the boot configs:

1. **Manifests** are generated from install-config (network, DNS, ingress, machine sets, and more).
2. **Ignition configs** are generated:
   - `bootstrap.ign`: large file with everything the bootstrap node needs to start the control plane (certificates, manifests, and the bootstrap services).
   - `master.ign`: small file that tells a control plane node to fetch its full configuration from the Machine Config Server on the bootstrap node.
   - `worker.ign`: small file that tells a worker node to fetch its config from the Machine Config Server.

These files are stored in your install directory (`~/ocp-ha-install`).

> **Important:** Ignition files contain certificates. Red Hat docs note that these certificates are valid for 24 hours, and recommend using the Ignition files within about 12 hours of generation. Do not reuse old Ignition files from a previous attempt.

Phase 2: Provision AWS infrastructure

The installer creates the network and compute resources. On AWS, this is done with Terraform (older installer versions required a separate `terraform` binary; check your installer version).

Resource	Purpose
VPC, subnets (per AZ), route tables, internet gateway, NAT gateways	Network for all nodes
Security groups	Allow cluster traffic (for example, 6443 for API, 22623 for MCS, etcd ports)
IAM roles and instance profiles	Permissions for control plane and worker nodes
S3 bucket	Stores `bootstrap.ign` so the bootstrap node can download it
API load balancer (external and internal)	Front door for the Kubernetes API (port 6443) and MCS (port 22623)
Ingress load balancer	Routes application traffic (ports 80 and 443) to worker nodes
Route 53 public records	`api.<cluster>.<base-domain>` and `*.apps.<cluster>.<base-domain>`
Route 53 private hosted zone	Internal name resolution, so nodes can reach `api-int`
EC2 instances	1 bootstrap + 3 control plane nodes, booted from the RHCOS AMI

Worker EC2 instances are **not** created in this phase. They are created later by the Machine API (Phase 6).

Phase 3: Nodes boot and Ignition runs

On first boot, each RHCOS machine runs **Ignition**:

- Sets hostname, network, and disk layout.
- Writes certificates, kubelet configuration, and container runtime (CRI-O) configuration.
- Starts the services needed for the node's role.

Control plane nodes contact the Machine Config Server on the bootstrap node to get the rest of their configuration.

Phase 4: Bootstrap creates the control plane

This is the core trick of OpenShift installation. There is a chicken-and-egg problem: you need a control plane to create a control plane. The installer solves it with a **temporary bootstrap machine**.

Steps, in order:

1. **Bootstrap starts a temporary control plane.** Services on the bootstrap node (run by `bootkube.sh` as `bootkube.service`) start a temporary etcd and a temporary Kubernetes API server.
2. **Control plane nodes fetch their config** from the bootstrap node's Machine Config Server (port 22623) and finish booting.
3. **etcd cluster forms.** The control plane nodes join etcd together with the bootstrap node.
4. **Temporary control plane schedules the production control plane** onto the three control plane nodes.
5. **Temporary control plane shuts down.** The production control plane takes over.
6. **Bootstrap injects OpenShift components** (such as the Cluster Version Operator) into the new control plane. The `bootkube` service then reports complete.

The installer watches this from your laptop with:

```bash
./openshift-install --dir ~/ocp-ha-install wait-for bootstrap-complete --log-level=info
```

When bootstrap is complete, the installer says it is safe to remove the bootstrap resources. It then:

- Removes the bootstrap instance from the load balancer.
- Terminates the bootstrap EC2 instance.
- Deletes the bootstrap Ignition file from S3.

Phase 5: Cluster Version Operator rolls out cluster operators

Now the production control plane runs. The **Cluster Version Operator (CVO)**:

- Reads the release image (the list of all components for your OpenShift version).
- Creates the cluster operators in dependency order (for example, networking and DNS first, then ingress, authentication, and console).
- Reports progress in `ClusterVersion` and `ClusterOperator` objects.

The **Machine Config Operator (MCO)** also starts. It applies OS-level configuration to nodes through `MachineConfig` objects. Some of these changes can cause nodes to reboot.

You can watch progress with:

```bash
oc get clusterversion
oc get co
```

Phase 6: Worker nodes are created and join

The **Machine API** uses **MachineSets** (created from install-config during Phase 1) to:

1. Call AWS to create the worker EC2 instances.
2. Boot each worker from the RHCOS AMI with `worker.ign`.
3. Worker fetches its config from the Machine Config Server.
4. The worker sends **Certificate Signing Requests (CSRs)** to join the cluster.

Pending CSRs must be approved before the worker becomes `Ready`. On IPI installs, the machine approver approves these automatically. If workers stay `NotReady`, check CSRs:

```bash
oc get csr
oc get nodes
```

Phase 7: Finish

The installer runs:

```bash
./openshift-install --dir ~/ocp-ha-install wait-for install-complete --log-level=info
```

This waits until:

- The API server is initialized.
- The `openshift-console` route exists.
- Cluster operators settle.

Then it prints the console URL and writes:

- `auth/kubeconfig`: login file for `oc`
- `auth/kubeadmin-password`: password for the `kubeadmin` user

4. Sample installer output (format)

This is the typical shape of the installer log. Exact messages vary by version.

```
INFO Creating infrastructure resources...
INFO Waiting up to 20m0s for the Kubernetes API at https://api.ocp-
ha.aws.example.com:6443...
INFO API v1.xx.x up
INFO Waiting up to 30m0s for bootstrapping to complete...
INFO It is now safe to remove the bootstrap resources
INFO Destroying the bootstrap resources...
INFO Waiting up to 40m0s for the cluster at https://api.ocp-ha.aws.example.com:6443 to
initialize...
INFO Waiting up to 10m0s for the openshift-console route to be created...
INFO Install complete!
INFO Run 'export KUBECONFIG=.../auth/kubeconfig' to manage the cluster with 'oc'
INFO Login to the console with user: kubeadmin, password: xxxxx-xxxxx-xxxxx-xxxxx
```

5. Timeline (approximate)

Stage	Typical time
Input check and asset generation	1-2 min
AWS infrastructure (VPC, LBs, Route 53, EC2)	10-15 min
Bootstrap to control plane	15-25 min
Bootstrap removed, cluster operators rollout	10-20 min
Workers created and join	5-10 min
Total	**35-60 min**

6. How to watch the install (useful commands)

Run these in a second terminal while the installer is running.

bash

```
# Installer log (most useful first stop)
tail -f ~/ocp-ha-install/.openshift_install.log

# Once the API is reachable
export KUBECONFIG=~/ocp-ha-install/auth/kubeconfig
oc get nodes
oc get co
oc get csr
oc get clusterversion
```

If bootstrap is stuck, look at the bootstrap node's logs. Red Hat's installer provides a gather command for this. Check your installer version's help output:

```bash
./openshift-install gather bootstrap --help
```

---

7. What happens at the end

Item	Result
Bootstrap EC2 instance	Terminated
Bootstrap S3 Ignition file	Deleted
Bootstrap entry in API load balancer	Removed
Control plane (3 nodes)	Running, etcd has 3 members
Workers (3 nodes)	Created by Machine API, joined the cluster
Cluster operators	Available
Console	Reachable at `console-openshift-console.apps.<cluster>.<base-domain>`

---

8. Troubleshooting by phase

Phase	Symptom	Where to look
Input check	Installer exits immediately	Terminal output, `.openshift_install.log`
AWS provisioning	Errors like quota, permission, or instance type	`.openshift_install.log` (search for `Error`)
Bootstrap	"Waiting for bootstrapping to complete" never finishes	Bootstrap node logs (`bootkube` service), check API LB and DNS
Control plane	Masters not Ready	`oc get nodes`, `oc get co`, `oc debug node/<name>`
Cluster operators	Some operators `Degraded` or not `Available`	`oc get co`, `oc describe co <name>`, `oc logs -n <namespace>`
Workers	Workers stuck `NotReady` or missing	`oc get csr`, `oc get machines -n openshift-machine-api`, `oc describe machine <name>`
Finish	Console route missing	`oc get route -n openshift-console`

9. Verification status of this document

Concept	Status	Source
Installer runs `create manifests` / `create ignition-configs` style steps; `wait-for bootstrap-complete` and `wait-for install-complete`	Verified	OpenShift installer docs and examples (github.com/openshift/installer overview)
Bootstrap machine acts as temporary control plane; masters form etcd with bootstrap; temporary control plane hands off; bootstrap injects components	Verified	OpenShift installer overview (github.com/openshift/installer, docs/user/overview.md)
Three Ignition configs: bootstrap, master, worker	Verified	OpenShift installer overview
Masters fetch config from bootstrap's Machine Config Server; port 22623 used by MCS; port 6443 for API	Verified	Red Hat / OpenShift UPI docs (search snippets)
`bootkube.service` runs on bootstrap	Verified	Dell OpenShift 4.10 implementation guide
Installer says it is safe to remove bootstrap resources; console and kubeadmin shown at finish	Verified	Installer example output (github.com/christianh814 and geliu2016 installer docs)
Route 53 private hosted zone created for internal lookups on AWS	Verified	OpenShift AWS install doc example (github.com/geliu2016/installer)
Terraform used for AWS infrastructure in IPI	Verified for older installer versions. Newer installer versions may embed it.	Installer README (pkg.go.dev)
Ignition certificates valid 24 hours; use within 12 hours	Verified	Red Hat OpenShift 4.6 AWS UPI docs (Chinese translation of the English page)

Concept	Status	Source
Worker CSRs must be approved; IPI approves automatically	Partly verified. Approval step verified in older docs; auto-approval mechanism is general knowledge.	Older Red Hat install guide snippet; general knowledge
Machine API creates workers from MachineSets	General knowledge, consistent with Red Hat docs	General knowledge
CVO rolls out cluster operators	Verified	Installer/OpenShift docs (CVO role)
Timeline values	General experience, not from docs	General knowledge
`openshift-install gather bootstrap`	General knowledge. Check `--help` in your version.	General knowledge

For production or exam preparation, confirm each item against the Red Hat documentation for your exact OpenShift version.

---

10. Sources

- OpenShift installer overview (bootstrap process, Ignition configs, command flow): https://github.com/openshift/installer/blob/3a604a3f060fea1a3a8c87968ee8af2db89a35eb/docs/user/overview.md
- OpenShift 4 for Ops (bootstrap process step by step): https://people.redhat.com/mskinner/MPC2019/OpenShift-4-for-Ops.pdf
- OpenShift UPI documentation (ports 6443, 22623, etcd, bootstrap removal): https://github.com/wking/openshift-installer/blob/master/docs/user/zvm/install_upi.md
- OpenShift AWS install example (Route 53 private zone): https://github.com/geliu2016/installer/blob/master/docs/user/aws/install.md
- Red Hat OpenShift 4.6 AWS UPI docs (Ignition certificate expiry note): https://access.redhat.com/documentation/zh-cn/openshift_container_platform/4.6/html/installing_on_aws/installation-aws-user-infra-installation_installing-aws-user-infra

- Dell OpenShift 4.10 implementation guide (bootkube service, wait-for commands): https://infohub.delltechnologies.com/document_parser/crosslinks/chapter/e6e3a4b6Kg
- Installer README (Terraform note, create and destroy): https://pkg.go.dev/github.com/pacevedom/installer